

PULSE ARTWORK STUDIO AI

System Architecture & Technical Design Specification

Version 1.0 • Genesis / MVP

Flutter Web + Python/FastAPI + PostgreSQL + Redis/Celery + Object Storage

Purpose: Translate the approved PRD into an implementation blueprint for a scalable commercial application.

1. Architecture Goals

Primary goal

Build a responsive, secure and scalable artwork-processing platform where the Flutter client remains responsive while expensive image and AI workloads execute asynchronously.

Architectural principles

Modular services; API-first communication; asynchronous processing; immutable originals; versioned outputs; secure secrets; observable processing jobs; replaceable AI models.

2. High-Level Architecture

Client

Flutter Web provides responsive UI, navigation, editor experience, project management and API interaction.

API

FastAPI exposes authenticated REST endpoints and coordinates projects, files, jobs, entitlements and exports.

Workers

Celery workers consume Redis-backed jobs and perform CPU/GPU-intensive image and AI operations.

Data

PostgreSQL stores structured metadata. S3-compatible object storage stores original artwork, intermediate assets, previews and exports.

Flow

Flutter → FastAPI → Job Queue → Worker → Object Storage → Job Status → Flutter.

3. Frontend Architecture

Framework

Flutter Web with feature-first modular organization.

State management

Use a predictable state-management approach such as Riverpod or Bloc; the final choice will be locked during implementation.

UI layers

App shell, responsive layout, shared design system, feature modules, domain models, repositories and API clients.

Editor

Canvas and tool panels should be isolated from backend-specific implementation so the editor can evolve without coupling to processing providers.

4. Backend Architecture

Framework

Python + FastAPI.

Responsibilities

Authentication, authorization, project CRUD, artwork metadata, upload orchestration, processing-job creation, export orchestration, usage/entitlement checks, and API-level validation.

Service boundaries

Auth, Projects, Artworks, Processing, Exports, Subscriptions/Entitlements, Users, Audit/Usage.

API versioning

Prefix public APIs with /api/v1 to allow controlled future changes.

5. Asynchronous Processing

Why

AI upscaling, segmentation, vectorization and effect generation can take seconds or minutes and must not block HTTP requests.

Lifecycle

REQUESTED → QUEUED → PROCESSING → COMPLETED or FAILED.

Job data

Job ID, project ID, artwork ID, operation, parameters, status, progress, timestamps, error code, result asset ID.

User experience

Flutter polls or subscribes to job status and shows progress, cancellation where supported, retry and clear error messages.

6. AI & Image Processing Pipeline

Enhancement

Upscaling, denoise, sharpening, color and artifact correction.

Segmentation

Background removal and edge refinement.

Vector

Raster preprocessing → color reduction/path extraction → curve smoothing → SVG validation.

Effects

Halftone and embroidery visual simulation should be deterministic from saved parameters where practical.

Model abstraction

AI model adapters should sit behind stable internal interfaces so providers/models can be swapped without changing the product layer.

7. Storage Architecture

Object storage

Use S3-compatible storage for binary assets.

Database

Store asset ID, project ID, original filename, MIME type, dimensions, DPI metadata, checksum, storage key, status and timestamps.

Originals

Original uploads are immutable.

Derived assets

Every processing operation creates a new version/asset rather than overwriting the source.

Security

Use signed, time-limited URLs for private asset access.

8. Core Database Model

users

Identity, account status, timestamps.

projects

Owner, name, status, created/updated timestamps.

artworks

Project association, source/derived relationship, metadata, storage key.

processing_jobs

Operation, parameters, status, progress, errors, result asset.

exports

Format, dimensions, asset reference, status, timestamps.

entitlements

Plan, limits, feature access and usage rules.

audit_logs

Security-sensitive and administrative events.

Recommended relationships

User 1→N Projects; Project 1→N Artworks; Artwork 1→N Processing Jobs; Job 0→1+ Derived Assets/Exports.

9. Authentication & Authorization

Authentication

Email/password initially, with OAuth providers as future extensions.

Sessions

Use secure, HTTP-only cookie/session architecture or a carefully managed token strategy; exact implementation will be finalized during backend setup.

Authorization

Every project, artwork and job request must be authorized against the authenticated user/tenant.

Admin

Administrative actions use explicit role-based permissions.

10. API Design

Representative endpoints

POST /api/v1/auth/login; POST /api/v1/projects; GET /api/v1/projects; POST /api/v1/projects/{id}/artworks; POST /api/v1/processing/jobs; GET /api/v1/processing/jobs/{id}; POST /api/v1/exports.

Upload pattern

Prefer direct-to-object-storage upload using signed URLs for large files, followed by metadata confirmation to FastAPI.

Responses

Use consistent success/error envelopes and machine-readable error codes.

Documentation

FastAPI-generated OpenAPI documentation should be maintained and reviewed as part of CI.

11. Security Architecture

Required

TLS everywhere, secure password hashing, input validation, MIME/type verification, file-size limits, malware scanning where appropriate, rate limiting, CSRF protection where cookie-based auth is used, secure CORS, least privilege, secret management.

AI providers

Provider keys remain server-side only.

File isolation

Users must never be able to enumerate or access another user's private storage keys.

12. DTF Domain Rules

Quality assistance

The application may flag likely problems but must not claim guaranteed print results.

DPI

Store both source metadata and calculated effective resolution where possible.

Transparency

Distinguish actual alpha transparency from a simulated preview background.

Underbase

White-underbase visualization is a simulation and not a RIP replacement.

Printer presets

Design the schema so future Pulse printer profiles can define recommended dimensions and processing defaults.

13. Performance & Scalability

Web

Lazy-load heavy editor features, compress previews, avoid loading full-resolution assets until required.

Processing

Use queues and worker autoscaling as usage grows.

Storage

Use object storage/CDN delivery for previews and exports.

Database

Index owner/project/job/status fields and use connection pooling.

GPU

GPU workers may be introduced independently for advanced AI models.

14. Observability & Operations

Logs

Structured application and worker logs with correlation/request IDs.

Metrics

API latency, queue depth, processing duration, failures, storage usage and worker utilization.

Alerts

Repeated job failures, queue saturation, API error spikes and infrastructure health.

Audit

Track authentication events, entitlement changes and administrative actions.

15. Testing Strategy

Frontend

Unit, widget/component, integration and responsive-layout tests.

Backend

Unit, API, authorization, database and contract tests.

Workers

Processor-level tests using known artwork fixtures and expected metadata/output constraints.

Visual regression

Maintain representative artwork fixtures for editor and effect previews.

Security

Dependency scanning, secret scanning, upload abuse tests and authorization tests.

16. CI/CD & GitHub

Branches

main for production; develop for integration; feature/*, fix/*, release/* and hotfix/* as defined in the repository architecture.

CI

Lint, formatting, unit tests, integration tests, dependency/security checks and build verification.

CD

Build immutable versioned artifacts and deploy through controlled environments.

Reviews

Protected main branch; pull-request review required; CI must pass before merge.

17. Environments

Local

Docker Compose for PostgreSQL, Redis and supporting services; Flutter and FastAPI run locally.

Staging

Production-like environment with separate database, object storage bucket and secrets.

Production

Managed PostgreSQL, Redis, object storage, API service, worker fleet and monitoring.

Data separation

Never share production customer data with development or test environments.

18. MVP Non-Functional Requirements

Availability

Design for high availability, but exact SLA should be established after pilot usage data.

Recovery

Automated database backups and documented restore procedure.

Privacy

Customer artwork is private by default.

Maintainability

Feature modules and service boundaries must allow independent evolution.

Portability

Dockerized backend services reduce vendor lock-in.

19. Initial Implementation Sequence

Step 1

Create GitHub repository and branch protection.

Step 2

Create Flutter application shell and design system.

Step 3

Create FastAPI service with `/api/v1` and health endpoints.

Step 4

Add PostgreSQL migrations and core user/project/artwork models.

Step 5

Add secure object-storage upload flow.

Step 6

Add processing-job model, Redis and Celery worker.

Step 7

Implement first processor: image metadata + basic enhancement.

Step 8

Connect Flutter editor to upload → job → result workflow.

Step 9

Add background removal, vector, halftone and embroidery modules incrementally.

Step 10

Add DTF Inspector and commercial licensing foundations.

20. Architecture Risks & Mitigations

Risk: AI costs grow quickly

Use job quotas, model routing, caching and plan-based entitlements.

Risk: Large files strain infrastructure

Direct uploads, object storage, previews and asynchronous workers.

Risk: Vector/embroidery quality varies

Use deterministic preprocessing, quality thresholds and user-adjustable parameters.

Risk: Browser memory limits

Use thumbnails/previews in the editor and defer full-resolution operations to workers.

Risk: Vendor lock-in

Abstract AI providers and use S3-compatible storage.

21. Architecture Decision Summary

Area	Decision	Reason
Frontend	Flutter Web	Responsive desktop/tablet/mobile UI with future native-mobile reuse.
API	Python + FastAPI	Strong fit for AI/image-processing ecosystem and high-performance APIs.
Database	PostgreSQL	Reliable relational foundation for commercial multi-user data.
Queue	Redis + Celery	Separates long-running processing from web requests.
Files	S3-compatible object storage	Scalable binary storage without bloating the database.
Deployment	Docker + CI/CD	Repeatable environments and controlled releases.
Architecture	Modular monolith + workers initially	Avoids premature microservice complexity while preserving clear boundaries.

Next artifact

Database Schema & API Contract Specification v1.0 should be produced next. It will lock the actual table fields, indexes, enums, request/response schemas, authentication flow, upload protocol, processing-job contract and error model before implementation begins.